

Anisotropic Line Filter Reformulation with Custom CUDA Kernels

Feasibility Study & Optimization Proposal

March 2026

1 Motivation

The Line Filter (LF) as currently implemented applies $(2m + 1)$ independent WVF evaluations along a line perpendicular to the candidate edge direction, then combines them with Gaussian weights. Each WVF evaluation involves:

1. Gathering N_p neighbor intensities around a virtual pixel
2. Rotating coordinates into the local frame at angle θ_k
3. Building a design matrix $\mathbf{A}$ and applying a precomputed pseudoinverse $\mathbf{P}_{\theta_k}$

For a single pixel at one orientation, the total cost is:

$$C_{\text{LF}} = (2m + 1) \cdot N_p \cdot M \quad (1)$$

where $M = (d + 1) \frac{d+2}{2}$ is the number of Taylor coefficients. Across all orientations N_s and all interior pixels N , the total work is $O(N \cdot N_s \cdot (2m + 1) \cdot N_p \cdot M)$.

The current GPU implementation already fuses the $(2m + 1)$ WVF applications into a single batched matmul per orientation, but the memory access pattern remains suboptimal: each virtual pixel gathers its own N_p neighbors via scattered indexing, leading to redundant loads (neighboring virtual pixels share most of their circular neighborhoods).

This document proposes a mathematically equivalent reformulation that eliminates the virtual-point iteration entirely, along with custom CUDA kernels that exploit the resulting structure.

2 Current Implementation Analysis

2.1 WVF Core Operation

At pixel (X_0, Y_0) and orientation θ_k , the WVF computes:

$$\hat{\mathbf{c}}^{(k)} = \mathbf{P}_{\theta_k} \mathbf{f} \quad (2)$$

where $\mathbf{f} \in \mathbb{R}^{N_p}$ is the vector of neighbor intensities and $\mathbf{P}_{\theta_k} = (\mathbf{A}_{\theta_k}^\top \mathbf{A}_{\theta_k})^{-1} \mathbf{A}_{\theta_k}^\top \in \mathbb{R}^{M \times N_p}$. The gradient estimate is $\hat{f}_x = \mathbf{e}_1^\top \hat{\mathbf{c}}^{(k)}$, i.e., row 1 of $\mathbf{P}_{\theta_k}$ dotted with $\mathbf{f}$.

2.2 LF Extension

The LF evaluates the WVF at $(2m + 1)$ virtual positions along a line:

$$(X_j, Y_j) = (X_0 + j \cos \theta_k, Y_0 + j \sin \theta_k), \quad j \in \{-m, \dots, m\} \quad (3)$$

and computes the weighted response:

$$R_k = \sum_{j=-m}^m w_j \cdot \hat{f}_x^{(j,k)}, \quad w_j = \exp\left(-\frac{j^2}{2\sigma^2}\right) \quad (4)$$

where $\hat{f}_x^{(j,k)}$ is the WVF normal derivative at virtual pixel j under orientation k .

The best orientation is $k^* = \arg \max_k |R_k|$, and the final gradient magnitude is $|R_{k^*}|$.

2.3 GPU Implementation (Current)

The current `lf/_cuda.py` implementation:

1. Precomputes $\mathbf{P}_{\theta_k}$ for all N_s orientations (shared with WVF)
2. Extracts $\mathbf{p}_{\text{fx}}^{(k)} = \mathbf{P}_{\theta_k}[1, :] \in \mathbb{R}^{N_p}$ (row 1 only)
3. For each orientation θ_k :
 - Computes virtual pixel positions for all line offsets $((L, B)$ tensor)
 - Gathers neighbors for all virtual pixels: (L, B, N_p) tensor
 - Flattens to $(L \cdot B, N_p)$ and does matmul with $\mathbf{p}_{\text{fx}}^{(k)}$
 - Reshapes to (L, B) , applies Gaussian weights, sums along line dimension

Bottleneck: The gather step creates an $(L \times B \times N_p)$ tensor per orientation, with heavily overlapping memory reads. For typical parameters ($L = 15$, $B = 500000$, $N_p = 100$), this is 750M scattered float32 reads per orientation.

3 Proposed Reformulation

3.1 Key Insight: Fusing Virtual Points into a Single Gather

Since each virtual pixel at position (X_j, Y_j) reads from the **same** set of N_p relative offsets (the circular neighbors in global coordinates), the full set of pixels touched by all $(2m + 1)$ virtual WVF applications can be described as:

$$\mathcal{N}_k = \{(X_0 + j \cos \theta_k + \Delta x_i, Y_0 + j \sin \theta_k + \Delta y_i) : j \in \{-m, \dots, m\}, i \in \{1, \dots, N_p\}\} \quad (5)$$

Many of these positions overlap. Instead of gathering N_p values $(2m + 1)$ times, we can gather the **union** of all needed pixels once, then apply a single fused linear operator.

3.2 Anisotropic Weighted Least-Squares

The LF response at pixel (X_0, Y_0) for orientation θ_k is:

$$R_k = \sum_{j=-m}^m w_j \cdot \mathbf{p}_{\text{fx}}^{(k)} \cdot \mathbf{f}_j \quad (6)$$

where $\mathbf{f}_j \in \mathbb{R}^{N_p}$ are the neighbor intensities around virtual pixel j .

Expanding: let $\mathbf{f}_{j[i]} = I(X_0 + j \cos \theta_k + \Delta x_i, Y_0 + j \sin \theta_k + \Delta y_i)$ where $(\Delta x_i, \Delta y_i)$ are the circular neighbor offsets. Then:

$$R_k = \sum_{j=-m}^m \sum_{i=1}^{N_p} w_j \cdot p_i^{(k)} \cdot I(X_0 + \delta_{j,i}^x, Y_0 + \delta_{j,i}^y) \quad (7)$$

where $p_i^{(k)} = \mathbf{p}_{\text{fx}}^{(k)}[i]$ and $\delta_{j,i}^x = j \cos \theta_k + \Delta x_i$.

This is a **single** weighted sum over all $(2m + 1) \times N_p$ stencil positions with weights $\alpha_{j,i}^{(k)} = w_j \cdot p_i^{(k)}$:

$$R_k = \boldsymbol{\alpha}_k^\top \mathbf{g} \quad (8)$$

where $\boldsymbol{\alpha}_k \in \mathbb{R}^{(2m+1) \cdot N_p}$ is the fused stencil weight vector and $\mathbf{g}$ is the vector of all unique pixel intensities in the stencil footprint.

3.3 Deduplication of Stencil Positions

The stencil positions $\mathcal{N}_k$ may contain duplicates (different (j, i) pairs mapping to the same pixel after rounding). For each unique position in $\mathcal{N}_k$, we sum the corresponding $\alpha_{j,i}^{(k)}$ values. This produces a **compressed stencil**:

$$R_k = \sum_{\ell=1}^{N'_k} \tilde{\alpha}_{k,\ell} \cdot I(X_0 + \tilde{\delta}_\ell^x, Y_0 + \tilde{\delta}_\ell^y) \quad (9)$$

where $N'_k \leq (2m+1) \cdot N_p$ is the number of unique positions. In practice, significant overlap occurs for small neighbor radii and large m , so N'_k can be substantially smaller.

3.4 Equivalence to Anisotropic Kernel

This reformulation reveals that the LF at orientation θ_k is mathematically equivalent to a **fixed linear filter** (convolution with a pre-computed kernel) whose support is an elongated, roughly elliptical region aligned with θ_k . The kernel weights $\tilde{\alpha}_{k,\ell}$ encode both the polynomial fitting (via $\mathbf{p}_{\text{fx}}$) and the Gaussian line averaging (via w_j).

This is equivalent to defining an anisotropic weight matrix $\mathbf{W}_k$ and solving:

$$\hat{\mathbf{c}}^{(k)} = (\mathbf{A}_k^\top \mathbf{W}_k \mathbf{A}_k)^{-1} \mathbf{A}_k^\top \mathbf{W}_k \mathbf{f}_{\text{all}} \quad (10)$$

over the full elongated neighborhood, where $\mathbf{W}_k$ applies tighter weighting perpendicular to θ_k and broader weighting along θ_k .

4 Optimization Strategies

4.1 Strategy 1: Precomputed Stencil Convolution

Since the fused weights $\tilde{\alpha}_{k,\ell}$ depend only on orientation (not pixel position), we precompute N_s sparse stencils and apply them as sparse convolutions:

Advantages:

- Single gather per pixel per orientation (instead of L gathers)
- Stencil weights precomputed once on CPU, transferred to GPU
- Natural fit for `torch.nn.functional.conv2d` with custom kernels, or sparse gather-multiply-sum

Complexity: $O(N \cdot N_s \cdot N'_k)$ vs current $O(N \cdot N_s \cdot L \cdot N_p)$

For typical values: $L = 15$, $N_p = 100$, deduplication gives $N'_k \approx 800$ – 1200 (vs $L \cdot N_p = 1500$), a 20–47% reduction in arithmetic. But the real win is **memory**: one gather instead of L .

4.2 Strategy 2: Dense Rectangular Kernel via conv2d

If we embed the sparse stencil into a dense rectangular kernel of size $(2R_y + 1) \times (2R_x + 1)$ (where R is the bounding box of the stencil), we can use `F.conv2d` directly:

- For each orientation θ_k , build a 2D kernel $\mathbf{K}_k$ of shape (h_k, w_k)
- Apply all N_s kernels as a grouped or batched convolution
- Result: (N_s, H, W) tensor of responses; take max across dim 0

Advantages:

- Leverages cuDNN’s highly optimized conv2d implementation
- No custom CUDA code needed for the first prototype
- Implicit memory coalescing and tiling from cuDNN

Disadvantages:

- Kernel can be large and sparse (many zeros inside the bounding box), wasting FLOPs
- Kernel size grows with m and neighbor radius

Estimated kernel size: For $m = 7$, neighbor radius ≈ 6 (for $N_p = 100$), the bounding box is roughly 20×20 pixels. For $m = 14$, it grows to $\approx 34 \times 34$.

4.3 Strategy 3: Custom CUDA Kernels

For maximum performance, we write custom CUDA kernels (via Triton or raw CUDA/C++):

4.3.1 Kernel 1: Fused Stencil Gather-Multiply-Sum

// Per-pixel kernel: one thread per (pixel, orientation) pair
 // or one warp per pixel across all orientations

```
__global__ void aniso_lf_kernel(
    const float* __restrict__ image,      // (H, W)
    const int* __restrict__ offsets,      // (Ns, N'_max, 2) stencil offsets
    const float* __restrict__ weights,    // (Ns, N'_max) stencil weights
    const int* __restrict__ n_unique,     // (Ns,) actual count per orient.
    float* __restrict__ grad_mag,        // (H, W)
    int* __restrict__ grad_idx,          // (H, W)
    int H, int W, int Ns, int N_max
) {
    int px = blockIdx.x * blockDim.x + threadIdx.x;
    int py = blockIdx.y * blockDim.y + threadIdx.y;
    if (px >= W || py >= H) return;

    float best = 0.0f;
    int best_k = 0;
    for (int k = 0; k < Ns; k++) {
        float response = 0.0f;
        int nk = n_unique[k];
        for (int i = 0; i < nk; i++) {
            int dy = offsets[k * N_max * 2 + i * 2];
            int dx = offsets[k * N_max * 2 + i * 2 + 1];
            float w = weights[k * N_max + i];
            response += w * image[(py + dy) * W + (px + dx)];
        }
        float abs_r = fabsf(response);
        if (abs_r > best) { best = abs_r; best_k = k; }
    }
    grad_mag[py * W + px] = best;
    grad_idx[py * W + px] = best_k;
}
```

Optimization opportunities:

- **Shared memory tiling:** Load image tile + halo into shared memory; stencil reads hit shmem instead of global
- **Warp-level reduction:** Distribute orientations across warp lanes; warp-reduce for max
- **Texture memory:** Use CUDA texture cache for 2D spatial locality of image reads
- **Register blocking:** Unroll inner stencil loop for known N'_k sizes

4.3.2 Kernel 2: Triton Implementation

Triton provides a higher-level alternative with automatic tuning:

```

@triton.jit
def aniso_lf_triton(
    image_ptr, offsets_ptr, weights_ptr,
    out_mag_ptr, out_idx_ptr,
    H, W, Ns, N_max,
    BLOCK_X: tl.constexpr, BLOCK_Y: tl.constexpr,
):
    px = tl.program_id(0) * BLOCK_X + tl.arange(0, BLOCK_X)
    py = tl.program_id(1) * BLOCK_Y + tl.arange(0, BLOCK_Y)
    # ... gather from image, dot with weights, reduce across Ns

```

Triton advantages:

- Auto-tuning of block sizes and memory access patterns
- Fused operations without manual shared memory management
- Easier to iterate on during research

4.4 Strategy 4: Square Neighborhood

Replace `get_circular_neighbors` with a square grid:

$$\mathcal{S}(r) = \{(\Delta x, \Delta y) : |\Delta x| \leq r, |\Delta y| \leq r, (\Delta x, \Delta y) \neq (0, 0)\} \quad (11)$$

giving $N_p = (2r + 1)^2 - 1$ neighbors.

Advantages:

- Contiguous memory access: neighbors lie in consecutive rows, enabling coalesced loads
- No distance computation for membership testing
- Row-major alignment: for a fixed Δy , all Δx values are consecutive in memory
- Better GPU occupancy: rectangular tiles map naturally to thread blocks
- Deterministic N_p for a given radius (no rounding ambiguity)

Impact on mathematics:

- The Taylor design matrix $\mathbf{A}$ changes (different coordinate values), but the pseudoinverse formulation is identical
- Condition numbers may actually improve: square grids provide more uniform angular coverage at corners vs. circles which undersample diagonals
- The anisotropic stencil becomes a rectangle-of-squares instead of rectangle-of-circles; the bounding box is tighter

Benchmark plan: Run ODS comparison with both neighborhood shapes at matched N_p counts.

4.5 Strategy 5: Reduced Orientation Count via Interpolation

Instead of evaluating all N_s orientations independently, evaluate a coarse set and interpolate:

1. Compute responses at $\frac{N_s}{q}$ evenly-spaced orientations
2. Fit a smooth function (e.g., trigonometric polynomial) to the orientation-response curve
3. Find the maximum analytically or via Newton refinement

Since the response $R(\theta)$ is typically smooth (it is a linear combination of image values with smoothly-varying weights), a 3:1 or 4:1 reduction may be feasible without ODS loss.

4.6 Strategy 6: Half-Precision (FP16/BF16) Computation

The stencil gather-multiply-sum is memory-bound, not compute-bound. Using FP16:

- Halves memory bandwidth for image reads (the bottleneck)

- Enables Tensor Core usage for the matmul path (if we keep the matrix formulation)
- Accumulation in FP32 avoids precision loss

Expected speedup: 1.5–2x on memory-bound kernels (bandwidth doubling minus overhead).

4.7 Strategy 7: Multi-Scale Pyramid

Apply the anisotropic LF at multiple image scales and merge:

- Coarse scales provide low-frequency edge context cheaply
- Fine scale provides localization
- Total work with a factor-2 pyramid: $\sum 4^{-k} \approx 1.33x$ single-scale cost for k levels

This is complementary to the other optimizations and may improve ODS independently.

5 Benchmarking Plan

5.1 Baseline Measurements

Current best ODS scores from full-dataset ablation:

Dataset	WVF ODS	LF ODS	Best WVF Config	Best LF Config
BSDS500	0.682	0.671	$N_p=100, N_s=4, d=2$	$N_p=250, N_s=18, d=4, m=2$
BIPED v1	0.812	0.802	$N_p=25, N_s=4, d=2$	$N_p=100, N_s=18, d=4, m=1$
BIPED v2	0.830	0.819	$N_p=25, N_s=4, d=2$	$N_p=100, N_s=18, d=4, m=1$
UDED	0.878	0.879	$N_p=100, N_s=4, d=2$	$N_p=100, N_s=18, d=4, m=2$

5.2 Evaluation Protocol

1. **Correctness:** Verify that the anisotropic reformulation produces identical (within FP tolerance) gradient maps to the current LF implementation on synthetic test images
2. **ODS/OIS:** Run full-dataset evaluation on all 4 datasets using the BSDS500 protocol (1001 thresholds, 3px match radius)
3. **Runtime:** Wall-clock comparison on standardized image sizes (481×321 for BSDS, 1280×720 for BIPED) across:
 - Current LF CUDA implementation
 - Anisotropic LF via `conv2d` (Strategy 2)
 - Anisotropic LF via custom CUDA/Triton kernel (Strategy 3)
4. **Memory:** Peak VRAM usage comparison
5. **Scaling:** Runtime vs. m (half-width) and N_p (neighbor count) curves

5.3 Success Criteria

- **Correctness:** Max absolute difference $< 10^{-4}$ vs reference implementation
- **Speed:** $\geq 2x$ wall-clock speedup over current LF CUDA at matched parameters
- **Quality:** ODS within 0.005 of current best (should be identical modulo FP order)

6 Implementation Roadmap

6.1 Phase 1: Stencil Precomputation (CPU/NumPy)

- Implement `build_anisotropic_stencil(theta, half_width, np_count, order)` that returns deduplicated offsets and weights
- Validate against current LF on single pixels
- Measure stencil sizes N'_k across parameter ranges

6.2 Phase 2: PyTorch conv2d Prototype

- Embed stencils into dense kernels, apply via `F.conv2d`
- Full-image correctness check
- Initial runtime comparison

6.3 Phase 3: Custom CUDA/Triton Kernel

- Implement sparse stencil kernel with shared memory tiling
- Auto-tune block sizes
- Benchmark against conv2d path

6.4 Phase 4: Square Neighborhood Variant

- Implement `get_square_neighbors` in `core/taylor.py`
- Re-run stencil construction with square support
- ODS comparison: circle vs. square at matched N_p

6.5 Phase 5: Additional Optimizations

- FP16 image storage with FP32 accumulation
- Orientation interpolation (coarse-to-fine)
- Multi-scale pyramid integration

6.6 Phase 6: Full Evaluation

- Complete ablation on all 4 datasets
- Statistical significance testing (Wilcoxon signed-rank)
- Runtime scaling curves
- Final report with figures

7 Risk Assessment

Risk	Likelihood	Mitigation
FP precision differences change ODS	Low	Use FP64 reference; only switch to FP16 after validating FP32
conv2d kernels too large/sparse	Medium	Fall back to sparse stencil kernel (Strategy 3)
Stencil deduplication savings < expected	Low	Even without dedup, single-gather beats L-gather
Triton kernel slower than cuDNN conv2d	Medium	Keep both paths; use whichever wins per config
Square neighborhood hurts ODS	Low	Optional; circle remains default if quality degrades